

1. Sintesi e contesto

Scopo del documento. Questa relazione verifica il lavoro presente sul branch main della repo giutala/ola.git e collega la soluzione dei primi due requisiti al materiale teorico caricato.

Repository controllata: <https://github.com/giutala/ola.git>. Commit verificato su main: 0da2a6a8835e108d26303daf46bcb20b2c04c6b8, autore Filippo Gulli, messaggio 'Fix req2 feasible joint actions', data 12 maggio 2026.

Verdetto breve: il Requirement 1 risulta implementato in modo coerente con la consegna; il Requirement 2, dopo il commit 0da2a6a, risolve il problema principale della fattibilit  delle azioni congiunte nel conflict graph.

La consegna definisce un problema di online learning applicato a campagne pubblicitarie. Un'agenzia deve scegliere dinamicamente bid per una o piu' campagne, osservando nel tempo gli esiti delle aste e rispettando un budget.

I parametri centrali sono T , il numero di campagne N , il valore v_i della vittoria nella campagna i , un insieme discreto di bid possibili e il budget complessivo B .

Il modello d'asta e' una first-price auction: se il bid b_i supera il massimo bid concorrente m_i , l'agenzia vince, ottiene utilita' $v_i - b_i$ e paga costo b_i . Quindi reward e costo sono accoppiati.

La difficolta' didattica e' che il learner non conosce prima la distribuzione dei massimi bid concorrenti. Deve esplorare per stimarla, ma anche sfruttare le stime per ottenere utilita' elevata.

Questo e' esattamente il terreno dei bandit stocastici: feedback parziale, incertezza sulle reward distribution, e obiettivo di regret sublineare rispetto al benchmark clairvoyant.

2. Requirement 1 in una frase

Il Requirement 1 chiede una singola campagna in ambiente stocastico. La repo implementa SingleCampaignEnv, UCB1BiddingAgent e UCBLikeBiddingAgent.

SingleCampaignEnv genera i bid concorrenti come variabili uniformi indipendenti e usa il massimo come soglia da battere. Questo realizza una distribuzione del highest competing bid.

UCB1BiddingAgent tratta ogni bid discreto come un braccio bandit e ignora il budget. Serve come baseline: impara quale bid e' redditizio ma non controlla la spesa.

UCBLikeBiddingAgent estende UCB1 con stime separate di utilita' e costo; sceglie una distribuzione sui bid risolvendo un linear program con vincolo di costo medio $\rho = B/T$.

Questa architettura aderisce alla consegna: ambiente stocastico, strategia UCB1 senza budget, e strategia UCB-like con gestione esplicita del budget.

3. Ambiente stocastico del Requirement 1

Nel codice, per ogni round vengono generati $n_competitors$ bid indipendenti uniformi in $[0,1]$. Il massimo m_t e' la soglia competitiva dell'asta.

La teoria spiega che un problema stocastico ha distribuzioni fisse associate alle azioni. Qui l'azione e' il bid, e la sua reward attesa dipende dalla probabilita' $P(b \geq m_t)$.

La probabilita' di vincita non e' stimata con euristiche arbitrarie nel benchmark: viene calcolata con la CDF $Beta(k,1)$, coerente con il massimo di k variabili uniformi.

L'ambiente restituisce $f_t = (value - bid)$ se il learner vince, altrimenti 0; restituisce $c_t = bid$ se vince, altrimenti 0. Cosi' utility e pagamento sono osservabili dopo l'azione.

Questa scelta collega direttamente l'implementazione alla parte teorica su reward distribution, pseudo-regret e confronto con un ottimo stocastico ben definito.

4. UCB1 come baseline

La teoria presenta UCB1 come algoritmo che bilancia exploration ed exploitation tramite ottimismo sotto incertezza. Un braccio poco esplorato riceve un bonus di confidenza.

Nel codice UCB1BiddingAgent mantiene `average_rewards` e `N_pulls` per ogni bid. Prima prova ogni bid, poi sceglie il bid con empirical mean piu' confidence bonus.

Questa e' una trasposizione naturale: ogni bid discreto e' un arm; la reward e' l'utilita' realizzata nell'asta. La discretizzazione rende il problema compatibile con MAB classico.

Il limite della baseline e' intenzionale: UCB1 massimizza reward ma non ha una variabile di costo. Percio' puo' consumare troppo budget, soprattutto quando il budget e' tight.

Nel report sperimentale questo confronto e' importante: mostra perche' il budget non e' un dettaglio implementativo, ma una componente strutturale del problema.

5. UCB-like con vincolo di budget

UCBLikeBiddingAgent e' il cuore del Requirement 1. Mantiene stime ottimistiche dell'utilita' e pessimistiche del costo: f_ucb e c_lcb .

La logica e' coerente con la teoria del budgeted bidding: quando la spesa e' vincolata, non basta sapere quale bid rende di piu'; bisogna sapere quanto costa in media.

L'algoritmo risolve un LP: massimizzare l'utilita' stimata sotto il vincolo che il costo stimato sia al massimo $\rho = B/T$, con una distribuzione sui bid che somma a 1.

Questo trasforma l'azione deterministica in una politica randomizzata. La randomizzazione e' naturale nei bandit con vincoli, perche' permette di rispettare budget medi e continuare a esplorare.

Il codice include anche lo stop quando il budget residuo e' minore di 1, evitando di continuare ad acquistare impression quando la risorsa e' esaurita.

6. Clairvoyant e pseudo-regret

La teoria definisce il regret come distanza dal benchmark ottimo. In ambiente stocastico, il benchmark ha senso perche' esiste una distribuzione fissa delle ricompense.

Nel Requirement 1 il clairvoyant e' `compute_clairvoyant_single`: usa le probabilita' vere di vincita e risolve un LP analogo a quello dell'agente, ma senza incertezza statistica.

Il pseudo-regret cumulato e' calcolato come somma nel tempo di `opt_utility_per_round - utility_t`. Questo e' coerente con la definizione teorica di pseudo-regret.

Il confronto non e' contro una policy impossibile nel modello: e' contro la migliore distribuzione sui bid che rispetta il budget medio nel mondo stocastico noto.

Questa scelta rende interpretabili i grafici: se il regret cresce sublinearmente, il learner si avvicina al comportamento del clairvoyant.

7. Esperimenti del Requirement 1

`run_req1.py` esegue due scenari: budget generous, con $\rho = 0.10$, e budget tight, con $\rho = 0.04$. Questa distinzione è didatticamente forte.

Nel budget generous, il vincolo tende a essere non binding. UCB1 e UCB-like dovrebbero comportarsi in modo simile perché la spesa non limita davvero la strategia.

Nel budget tight, UCB1 può sovraspendere o consumare budget presto, mentre UCB-like controlla il costo medio e smette quando il budget residuo è insufficiente.

I plot di regret e costo cumulato collegano l'implementazione alla teoria: performance non si misura solo in reward, ma anche nella fattibilità rispetto alla risorsa scarsa.

In sintesi: il Requirement 1 è risolto perché contiene ambiente, baseline, algoritmo budget-aware, clairvoyant e metriche coerenti.

8. Requirement 2 in una frase

Il Requirement 2 generalizza il problema a piu' campagne in ambiente stocastico e richiede una strategia Combinatorial-UCB con vincolo di budget.

La consegna aggiunge una componente decisiva: il conflict graph. Ogni nodo e' una campagna; un arco indica che due campagne sono mutuamente esclusive.

Questo significa che due campagne incompatibili non possono ricevere un bid nello stesso round. La fattibilita' deve valere nell'azione realizzata, non solo in media.

La repo implementa MultiCampaignEnv, compute_clairvoyant_multi e CombinatorialUCBAgent. Il commit 0da2a6a modifica proprio questi punti.

Il problema risolto dal commit e' il passaggio da marginali indipendenti per campagna a una distribuzione su joint actions gia' fattibili.

9. Il bug precedente nel Requirement 2

Prima del commit 0da2a6a, l'agente risolveva un LP con variabili x_{ik} marginali: una variabile per campagna i e bid k .

Il LP poteva imporre vincoli del tipo 'la massa totale su due campagne incompatibili e' al massimo 1'. Tuttavia, dopo il LP, il codice campionava indipendentemente per ogni campagna.

Questa indipendenza rompeva il significato del vincolo: anche se le marginali rispettavano il conflict graph in aspettativa, una realizzazione poteva selezionare entrambe le campagne incompatibili.

Il problema era ancora piu' serio nella fase greedy iniziale, dove si poteva scegliere il miglior bid per ogni campagna ignorando gli archi di conflitto.

Quindi il vecchio codice non garantiva la frase centrale della consegna: campagne non compatibili non possono essere biddate nello stesso round.

10. La correzione del commit nerd53

Il commit 0da2a6a introduce `self.joint_actions` in `CombinatorialUCBAgent`. Le azioni congiunte sono tuple: per ogni campagna contengono un indice di bid oppure -1 per astensione.

La funzione `_build_joint_actions` usa backtracking e un controllo di compatibilit . Quando una campagna   attiva, una campagna collegata da un arco non puo' essere aggiunta.

Il LP non produce piu' marginali indipendenti. Produce gamma, una distribuzione di probabilit  sulle joint actions fattibili.

Quando l'agente deve agire, campiona un solo indice da gamma e applica quella joint action. Poich  ogni azione nel supporto   fattibile, ogni round realizzato   fattibile.

Ho verificato staticamente 288 joint actions per i parametri della repo: zero violazioni degli archi (0,1) e (2,3).

11. Perche' la fix risolve il requisito

Il punto matematico e' sottile ma importante. Un vincolo su marginali non basta quando il sistema richiede fattibilita' round-by-round.

Esempio: una distribuzione marginale puo' dire che campagna 0 e campagna 1 hanno ciascuna probabilita' 0.5 di essere attive. Se campionate indipendentemente, possono attivarsi insieme con probabilita' positiva.

Il conflict graph invece richiede probabilita' zero di attivazione simultanea per ogni arco. Questo si ottiene solo campionando da uno spazio di azioni che esclude gia' quelle combinazioni.

La correzione applica questa idea. Il vincolo di conflitto non e' un vincolo approssimato sul valore atteso: e' incorporato nella definizione dello spazio azionario.

Per questo il commit nerd53 risolve il problema specifico del Requirement 2 relativo alle feasible joint actions.

12. Ambiente multi-campagna

MultiCampaignEnv modella N aste first-price indipendenti con budget condiviso. Ogni campagna ha il proprio massimo bid concorrente e la propria probabilita' di vittoria.

Dopo il commit, l'ambiente non corregge piu' a posteriori due vittorie incompatibili scegliendo quella migliore. Invece rifiuta una bid vector non ammissibile con ValueError.

Questa scelta e' corretta dal punto di vista della specifica: se due campagne non possono essere biddate nello stesso round, l'ambiente deve trattare quell'azione come invalida.

L'ambiente diventa quindi un guardrail per l'agente. Se l'agente producesse ancora azioni non fattibili, il run fallirebbe subito invece di nascondere l'errore.

Ho eseguito una simulazione di 300 round con l'agente corretto: 0 violazioni e nessun errore dell'ambiente.

13. Clairvoyant multi-campagna

Anche `compute_clairvoyant_multi` e' stato aggiornato. Prima usava variabili piatte marginali; ora enumera le stesse azioni congiunte fattibili dell'agente.

Questo e' fondamentale per il regret. Se il benchmark potesse usare azioni non realizzabili o una rilassazione diversa, il confronto empirico sarebbe distorto.

Il nuovo clairvoyant risolve un LP su azioni congiunte: massimizza l'utilita' attesa totale, con vincolo di costo atteso per round al massimo ρ .

Poi restituisce anche le marginali per campagna, utili per interpretazione, ma l'ottimo e' calcolato sullo spazio corretto.

Così' agente e benchmark vivono nello stesso modello: stessa distribuzione stocastica, stesso budget, stesso conflict graph, stesso spazio di fattibilita'.

14. Collegamento ai combinatorial bandits

Il file teorico introduce i combinatorial bandits come estensione dei MAB in cui il learner seleziona un sottoinsieme di arms, detto superarm, a ogni round.

Nel Requirement 2, una joint action e' proprio un superarm strutturato: contiene una scelta di bid per alcune campagne e astensione per altre.

La compatibilita' del conflict graph definisce quali superarm sono ammissibili. Il budget aggiunge un vincolo globale sul costo atteso.

Combinatorial-UCB conserva l'idea di UCB: mantiene stime e confidence bonus per componenti elementari (campagna, bid), poi usa un oracle per scegliere il miglior superarm stimato.

La repo implementa questa architettura: stime semi-bandit per ogni coppia (i,k) , LP oracle per combinare le componenti e campionamento della joint action.

15. Feedback semi-bandit

Nel codice, dopo una joint action, l'agente aggiorna solo le coppie (campagna, bid) effettivamente attive. Le astensioni non aggiornano reward o costi.

Questo e' coerente con il feedback semi-bandit: quando seleziono un superarm, osservo i risultati delle sue componenti, non necessariamente di tutte le azioni alternative.

La teoria distingue full information e bandit feedback. Qui non siamo in full information: il learner non osserva cosa sarebbe successo con bid diversi o campagne non selezionate.

L'uso di UCB compensa proprio questa informazione parziale: le componenti poco esplorate ricevono valori ottimistici, quindi restano candidate per l'esplorazione.

Il Requirement 2, quindi, non e' solo 'piu' UCB1 in parallelo': e' un bandit combinatoriale con feedback parziale e vincoli strutturali.

16. Budget condiviso e LP oracle

Il budget nel Requirement 2 e' condiviso fra campagne. Una campagna con bid aggressivo puo' ridurre la capacita' di spendere sulle altre.

Per questo l'oracle deve ragionare sulla joint action intera: utilita' totale stimata e costo totale stimato. Il costo non e' per campagna isolata, ma aggregato.

Il LP dell'agente massimizza $\sum_a \gamma_a UCB_f(a)$, soggetto a $\sum_a \gamma_a LCB_c(a) \leq \rho$ e $\sum_a \gamma_a = 1$.

Questa e' la naturale estensione del LP single-campaign. Nel Requirement 1 la distribuzione e' sui bid; nel Requirement 2 la distribuzione e' sulle azioni congiunte.

Il legame teorico e' chiaro: UCB gestisce incertezza statistica, LP gestisce il vincolo di risorsa, combinatorial oracle gestisce la struttura delle azioni.

17. Regret e andamento atteso

La teoria indica che l'obiettivo degli algoritmi online è ottenere regret sublineare, così che il regret medio per round tenda a zero.

Nel codice, i grafici confrontano il regret cumulato con una curva di riferimento $O(\sqrt{T \log T})$. Questa non è una prova formale, ma è un controllo visivo sensato.

Per Requirement 1, la differenza fra budget generous e tight mostra come un vincolo attivo cambi l'andamento: controllare la spesa può aumentare regret iniziale ma preserva fattibilità.

Per Requirement 2, il regret ha senso solo dopo aver corretto lo spazio delle azioni. Prima, l'agente e il benchmark potevano ragionare su una rilassazione non coerente con il modello.

Dopo il commit, il regret misura apprendimento dentro il problema corretto: campagne compatibili, budget condiviso e ambiente stocastico.

18. Cosa resta da monitorare

La fix risolve il problema principale del Requirement 2, ma ci sono aspetti da monitorare se il progetto viene esteso.

Numero di azioni: enumerare tutte le joint actions funziona per N piccolo e bid set discreti. Se N cresce molto, lo spazio può diventare esponenziale e servirà un oracle più scalabile.

Budget effettivo: l'LP controlla costo atteso per round tramite LCB. Nei singoli run possono esistere fluttuazioni; i plot di costo cumulato sono quindi importanti.

Fallback greedy: ora è fattibile rispetto al conflict graph, perché sceglie la migliore joint action ammissibile. Resta però una fase esplorativa ottimistica, quindi va interpretata come parte dell'UCB.

Documentazione: il file docs/req1_req2_corrections.tex già riassume la correzione; questa relazione ne espande il collegamento teorico.

19. Verifiche eseguite

Verifica Git: main punta a 0da2a6a, autore Filippo Gulli, con modifica di agents.py, environments.py, experiments.py e docs/req1_req2_corrections.tex.

Verifica del diff: il commit rimuove il campionamento indipendente dalle marginali e introduce gamma su self.joint_actions.

Verifica statica: con $N=4$, $K_s=[9,6,10,7]$ e $\text{conflict_edges}=[(0,1),(2,3)]$, l'agente enumera 288 joint actions e nessuna viola gli archi di conflitto.

Verifica dinamica: simulazione di 300 round in MultiCampaignEnv con CombinatorialUCBAgent corretto, risultato 0 violazioni, costo cumulato 66.1, budget residuo 1533.9.

Verifica documentale: project-2.pdf richiede per Requirement 2 una joint distribution sui highest competing bids e una strategia Combinatorial-UCB con budget constraint.

20. Conclusione

Risposta alla domanda principale: si', il commit fatto da nerd53 su main risolve il problema del Requirement 2 relativo alle azioni congiunte fattibili.

La ragione e' precisa: il codice ora campiona direttamente da uno spazio di joint actions che esclude combinazioni incompatibili, quindi gli archi del conflict graph sono rispettati in ogni round realizzato.

Requirement 1 e Requirement 2 sono collegati dalla stessa idea teorica: apprendere in ambiente stocastico con feedback parziale, minimizzare pseudo-regret e usare UCB per bilanciare exploration ed exploitation.

Il passaggio da Requirement 1 a Requirement 2 aggiunge due livelli: un budget condiviso su piu' campagne e una struttura combinatoriale delle azioni. Per questo serve un LP oracle su azioni congiunte, non solo UCB separati.

La soluzione attuale e' quindi coerente con la consegna e con la teoria caricata: single-campaign UCB-like per il primo requisito, Combinatorial-UCB budget-aware su superarm fattibili per il secondo.